

Project TICKTOCK

BOB for Commerce - technical build flow

E-Commerce module only | FastAPI (Python) + Next.js | WhatsApp via OpenWA | v2 - July 2026

1. Scope

This document covers the **Commerce module only** - the insurance line is deliberately left for later. Everything here builds toward one outcome from the vision deck: *post once, sell everywhere*. Three pieces make that real:

BOB Page - the AI link-in-bio. A seller pastes a TikTok/Instagram link, BOB pulls the video, caption and thumbnail, and the seller adds only price and stock. Buyers see today's drop with live availability - no re-uploading.

BOB Agent - the WhatsApp close. One tap opens WhatsApp with the product already identified. BOB confirms stock, takes size and delivery, fires the M-Pesa STK push, and treats the Daraja callback as payment truth.

BOB Reach - consent-based marketing. Every sale ends with "Want new-drop alerts? Reply YES", and BOB later broadcasts restocks - paced, opt-in only, with STOP handling.

2. WhatsApp via OpenWA (design decision)

Because the official WhatsApp Cloud API path is hard right now (it needs a verified Meta Business Manager on the bonganabob.com domain - Gate 2 in the deck), the Commerce build uses **OpenWA** (open-wa.org) instead. OpenWA is a free, self-hosted WhatsApp HTTP API gateway: a Docker container that links a real WhatsApp number by QR scan and gives you a REST API to send and real-time webhooks to receive.

What this changes for us, in plain terms:

No Meta verification to start - you can build and pilot immediately by scanning a QR code with a WhatsApp number. Gate 2 no longer blocks the prototype.

Clean fit with FastAPI - OpenWA runs as its own service; our backend talks to it over HTTP using the official Python SDK (rmyndharis-openwa). It POSTs incoming messages to our /whatsapp webhook (HMAC-signed), and we call its REST API to reply.

The trade-off: it is unofficial - OpenWA is not affiliated with Meta and automating a personal number carries an account-ban risk if abused. This is manageable but must be designed for, not ignored (see section 8).

3. Tech stack

Layer	Choice	Why
Frontend	Next.js (React, TypeScript, Tailwind)	BOB Page + seller dashboard; fast public pages, easy Vercel deploy
Backend	FastAPI (Python)	Async webhooks, agent logic, Daraja + OpenWA clients
WhatsApp	OpenWA gateway (self-hosted, Docker)	REST API + webhooks; whatsapp-web.js/baileys engine; no Meta verification; Python SDK rmyndharis-openwa
Payments	M-Pesa Daraja (STK push + callback)	The payment truth for every confirmed order

Layer	Choice	Why
Database	PostgreSQL + SQLAlchemy + Alembic	Sellers, products, orders, consent; versioned migrations
Queue / cache	Redis	Reach broadcast pacing, background jobs, callback idempotency
AI	LLM API (agent replies)	Natural-language ordering; deterministic pricing stays in code
Hosting	Vercel (frontend) + a VPS for backend + OpenWA	OpenWA needs a persistent host to hold the WhatsApp session

4. Folder flow

Create this inside your desktop **Project TICKTOCK** folder. One repo with two apps (frontend/ and backend/), the openwa/ gateway service, a shared/ contract folder, and docs/. Build the backend for a feature first, then wire the Next.js UI to it.

```
Project-TICKTOCK/
|
+-- README.md           project overview + run instructions
+-- .env.example        every key the app needs (copy to .env)
+-- .gitignore
+-- docker-compose.yml   Postgres + Redis + OpenWA gateway
|
+-- frontend/           Next.js -- BOB Page + seller dashboard
|   +-- app/
|   |   +-- [handle]/page.tsx   public page: bob.link/mama-wanjiku
|   |   +-- dashboard/         seller: paste link, set price & stock
|   |   +-- api/               light BFF routes -> backend
|   |   +-- components/        product card, drop list, order button
|   |   +-- lib/               api client, shared TS types
|   |   +-- public/
|   |   +-- package.json
|   |
|   +-- backend/           FastAPI -- agent, webhooks, payments
|   |   +-- app/
|   |   |   +-- main.py         app entry, registers routers
|   |   |   +-- config.py       env vars, secrets
|   |   |   +-- db.py           SQLAlchemy engine + session
|   |   |   +-- api/
|   |   |   |   +-- products.py  scrape link, set price / stock
|   |   |   |   +-- whatsapp.py  OpenWA webhook receiver (HMAC verify)
|   |   |   |   +-- daraja.py    STK push + M-Pesa callback
|   |   |   |   +-- reach.py     broadcasts, consent, STOP
|   |   |   +-- agent/
|   |   |   |   +-- state.py     conversation state machine
|   |   |   |   +-- flows.py     order -> size -> delivery -> pay
|   |   |   |   +-- prompts.py   LLM prompts for the agent
|   |   |   +-- services/
|   |   |   |   +-- scraper.py    pull TikTok/IG video, caption, thumb
|   |   |   |   +-- whatsapp.py  OpenWA REST client (rmyndharis-openwa)
|   |   |   |   +-- mpesa.py     Daraja client (auth, STK, query)
|   |   |   |   +-- reach.py     pacing, warm-up, block-rate guard
|   |   |   +-- models/         seller, product, order, consent
|   |   +-- alembic/            database migrations
|   |   +-- tests/
|   |   +-- requirements.txt
|   |   +-- Dockerfile
|   |
|   +-- openwa/             self-hosted WhatsApp gateway (Docker)
|   |   +-- (runs OpenWA image; scan QR in its dashboard :2785 to link number)
|   |
|   +-- shared/
|   |   +-- schemas/         request/response contracts, both sides
|   |
|   +-- docs/
|   |   +-- technical-flow.docx  this document
```

5. How the project moves in VS Code

Read the diagram in section 6 as the spine: each milestone is a branch, and you only move down once the current one demos. Day to day inside VS Code:

Three services up - backend (uvicorn app.main:app --reload), frontend (npm run dev), and the OpenWA gateway + Postgres + Redis via docker compose up. OpenWA's dashboard is at <http://localhost:2785> - scan the QR once to link the number.

One branch per milestone - git checkout -b m1-bob-page. Commit at each green test; merge when the milestone's "done when" is met.

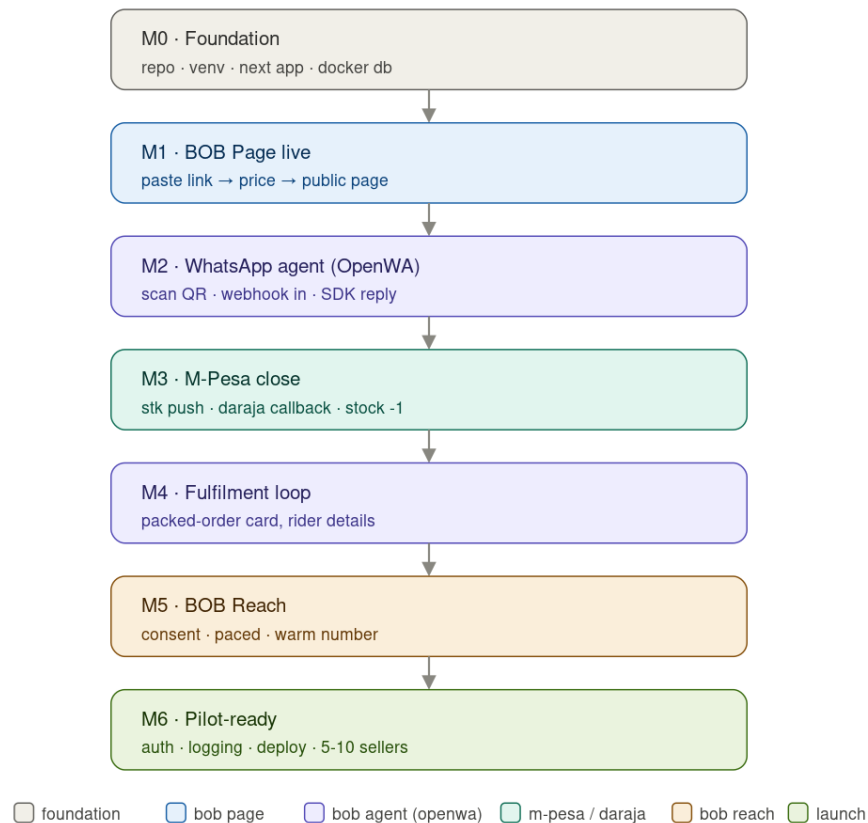
Feature loop - for each feature: model → migration → service → API route → test, then the UI. This keeps the data spine correct before anything talks to it.

Tunnel for callbacks - during M3, expose localhost with ngrok or cloudflared so Daraja can reach your callback URL. OpenWA runs locally, so its webhook can point straight at your backend.

Extensions: Python + Pylance + Ruff, ESLint + Prettier, Tailwind CSS IntelliSense, Docker, and a REST client (Thunder Client).

6. Build flow diagram

BOB Commerce — how the project moves in VS Code (M0 to pilot)



7. Milestones

Each milestone ends in something you can demo. Do not start the next until the “done when” is true. The right column maps to the roadmap in your vision deck.

Milestone	Build in VS Code	Done when (demo)	Deck phase
M0 - Foundation	Scaffold repo, git init, Python venv + FastAPI, Next.js app, docker-compose for Postgres + Redis + OpenWA, .env	All services boot; migrations run; /health returns 200	Phase 0
M1 - BOB Page	Product model; link scraper; dashboard form (paste link, price, stock); public [handle] page with live availability	Seller creates a page; public link shows item, price, and Available/SOLD	Phase 0
M2 - WhatsApp agent (OpenWA)	Run OpenWA container; scan QR to link number; receive HMAC webhook; wa.me deep link from page; agent identifies product + confirms stock; reply via SDK	Tapping “Order on WhatsApp” opens a pre-filled chat; BOB replies with the product and availability	Phase 0
M3 - M-Pesa close	Daraja sandbox; agent takes size + delivery; STK push; callback = payment truth; decrement stock; one-text SOLD	Sandbox payment completes; order marked paid; stock auto-updates to SOLD	Phase 0-1
M4 - Fulfilment	Packed-order card to seller; rider/delivery details	After payment, seller gets a packed card	Phase 1

Milestone	Build in VS Code	Done when (demo)	Deck phase
	to buyer; order status transitions	and the buyer gets rider details	
M5 - BOB Reach	Consent capture at checkout; consent registry; paced broadcast worker with simulated typing + rate limits; STOP handling	An opted-in buyer receives a restock broadcast; STOP removes them; sends stay within safe pacing	Phase 2
M6 - Pilot-ready	Seller auth; audit logging on every send; error handling; deploy frontend + backend + OpenWA on a VPS; warmed-up number	Live on a real number; a real seller runs one full order end to end	Phase 1 pilots

8. Keeping the WhatsApp number safe

Because OpenWA drives a real number, account safety depends entirely on messaging behaviour. Build these in from M2, and treat them as hard rules for BOB Reach (broadcasts carry the most risk):

Warm up new numbers - use the number normally for a while before automating; don't blast from a cold SIM.

Transactional first - BOB Agent replies to buyers who messaged first (low risk). Reach is opt-in only - no cold outreach, no scraped lists.

Pace and randomise - use OpenWA's simulated typing and randomised delays; keep strict rate limits on broadcasts.

STOP + block monitoring - honour STOP instantly and watch block/complaint rate; back off if it climbs.

Have a fallback - keep the code behind our own services/whatsapp.py interface so that if you later get Meta verification, you can swap OpenWA for the official Cloud API without touching agent logic.

9. Keys to collect before M2

OpenWA - a WhatsApp number to link by QR, an OpenWA API key, and a webhook HMAC secret. All self-generated - no Meta approval needed.

M-Pesa / Daraja - consumer key + secret, business shortcode, passkey, and a public callback URL. Start in the Safaricom sandbox.

Plus: a Postgres URL, a Redis URL, and an LLM API key. Keep them all in .env and never commit it.

10. Immediate next actions

1. Create the folder tree above inside Project TICKTOCK and git init.
2. Stand up M0: FastAPI /health, Next.js hello page, docker-compose with Postgres + Redis + OpenWA.
3. Link a WhatsApp number in the OpenWA dashboard and register a Daraja sandbox account.
4. Build M1 (BOB Page) - it is the most visible win and what you show the first pilot sellers.

Post once. Sell everywhere.